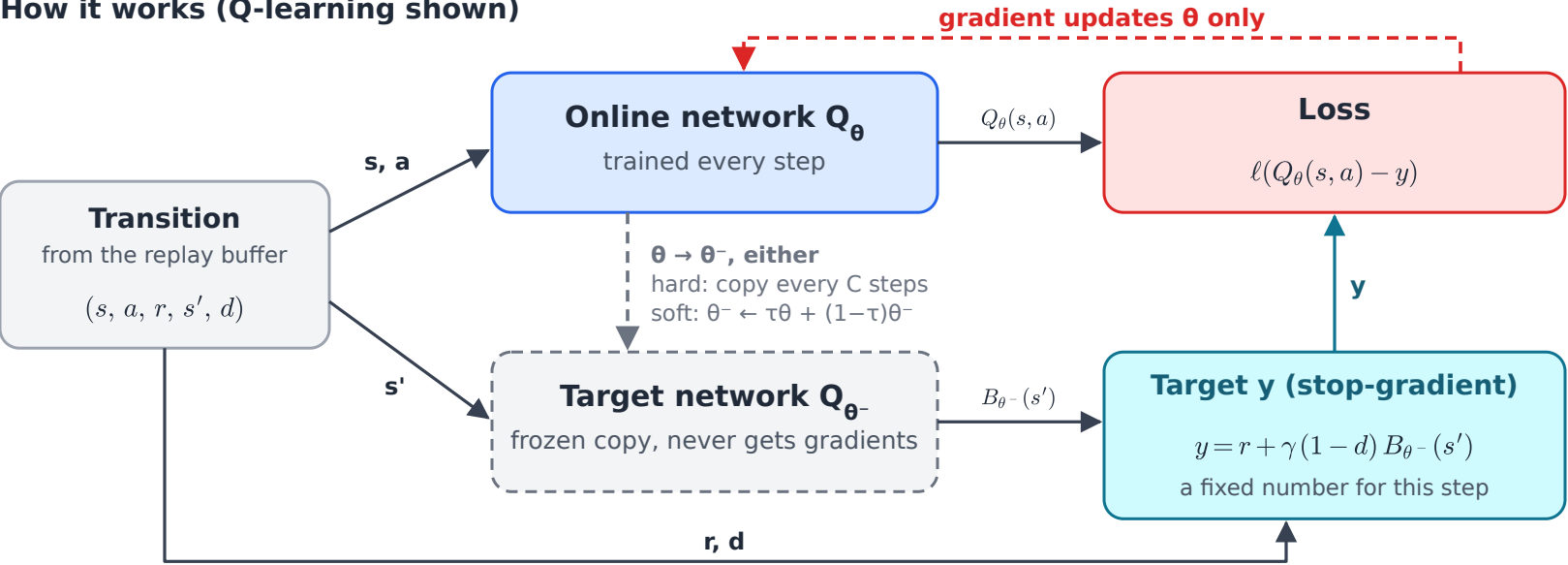


Target networks cheat sheet

A slowly updated copy of a network, used only to compute bootstrapped regression targets. What it is, why it helps, how it's updated, and which algorithms use it (page 2).

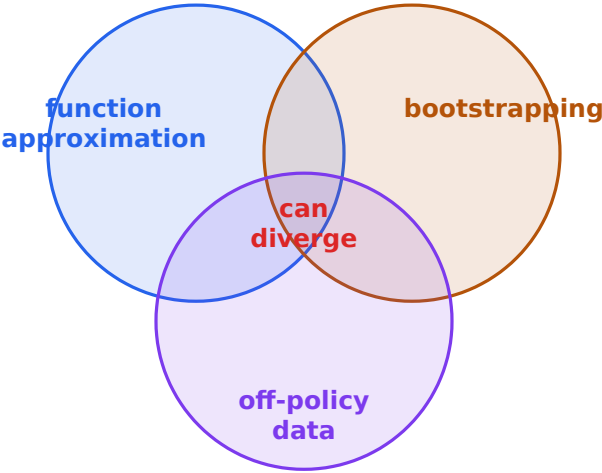
How it works (Q-learning shown)



The bootstrap term $B_{\theta^-}(s')$ in popular algorithms

DQN	$\max_{a'} Q_{\theta^-}(s', a')$	TD3	$\min_i Q_{\bar{\phi}_i}(s', \mu_{\bar{\theta}}(s') + \tilde{\epsilon})$
Double DQN	$Q_{\theta^-}(s', \arg \max_{a'} Q_\theta(s', a'))$	SAC v2	$\min_i Q_{\bar{\phi}_i}(s', \tilde{a}') - \alpha \log \pi_\theta(\tilde{a}' s')$
DDPG	$Q_{\bar{\phi}}(s', \mu_{\bar{\theta}}(s'))$	bars mark target copies. Actor-critic methods also keep a target actor (DDPG, TD3) or sample $\tilde{a}'$ from the current π (SAC).	

Why it's needed: the deadly triad

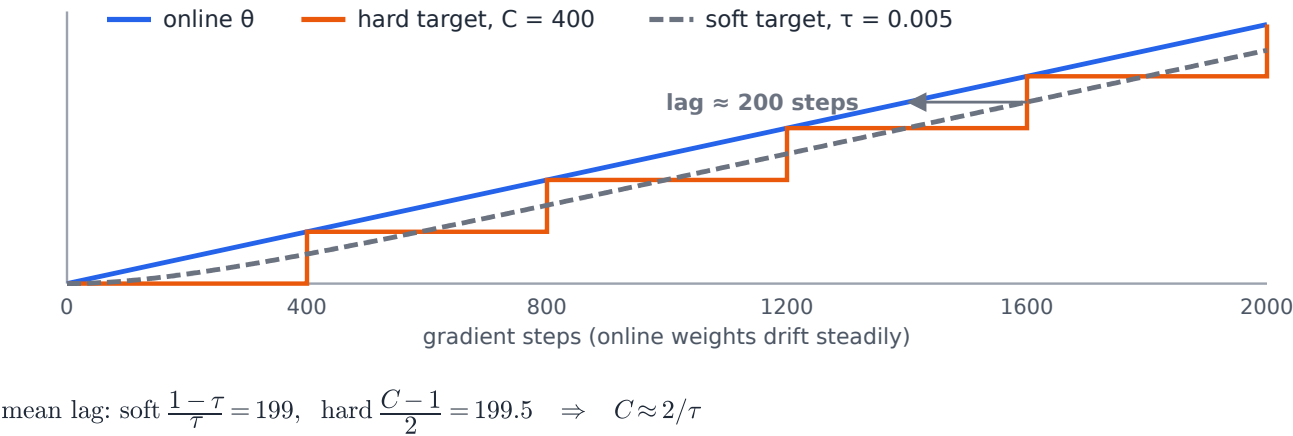


Deep Q-learning has all three (Sutton & Barto, ch. 11). Updating $Q(s, a)$ also moves $Q(s', \cdot)$: neighbouring states share weights, so the label y shifts with every step, and errors feed back through the max. A target network cuts this fast loop: for C steps (or roughly $1/\tau$), labels hold still.

Moving target vs. frozen target (schematic, not data)



Hard vs. soft updates tracking a drifting network (exact)



What a target network does, and doesn't, fix

- Does**
- ✓ Holds the regression labels still between updates
 - ✓ Breaks the fast feedback loop through shared weights
 - ✓ Turns training into a sequence of supervised fits (fitted Q-iteration)
 - ✓ Makes replayed, off-policy data and larger learning rates workable
- Doesn't**
- ✗ Remove the max's upward bias: use Double DQN or clipped double-Q
 - ✗ Fix offline distribution shift: that needs conservatism (CQL, IQL, TD3+BC)
 - ✗ Speed learning up: new value information reaches targets with a lag
 - ✗ Guarantee convergence: divergence gets rarer, not impossible

Do you need one?

- Likely yes, when all three hold:**
- the target bootstraps from the network you're training (Q-learning, TD critics)
 - that network is a neural net shared across states
 - data is replayed or off-policy, with many updates per sample
- Usually no, when:**
- targets are Monte Carlo returns (REINFORCE)
 - targets are computed once per fresh on-policy batch, then frozen (A2C, PPO)
 - there is no critic at all (GRPO, RLOO, DPO)
 - normalization is designed to replace it (PQN with LayerNorm, CrossQ with batch renorm)

Which algorithms use a target network?

Bars mark target copies (θ^- , $\bar{\phi}$, $\bar{\psi}$, $\hat{\theta}$). The last column is the term $B(s')$ in $y = r + \gamma(1 - d) \cdot B(s')$ unless noted. Hyperparameters are from the original papers.

Algorithm	Family	Has a target copy of	Update rule	Bootstrap term, or how targets are built	Note
Value-based, discrete actions					
DQN (2015)	Q-learning	Q network	hard, every 10^4 updates	$\max_{a'} Q_{\theta^-}(s', a')$	introduced it, with replay
Double DQN (2016)	Q-learning	Q network	hard	$Q_{\theta^-}(s', \arg \max_{a'} Q_{\theta}(s', a'))$	online picks, target evaluates
Dueling, PER	Q-learning	Q network	hard	as DQN or Double DQN	architecture / sampling changes
C51, QR-DQN, IQN	distributional	return distribution	hard	distribution of $r + \gamma Z_{\theta^-}(s', a^*)$	projected or quantile targets
Rainbow (2018)	Q-learning	distributional Q	hard	n-step + Double + distributional	six DQN extensions combined
R2D2, Agent57	recurrent, distributed	Q network	hard, periodic	n-step Double-Q with value rescaling	replayed sequences
Continuous control, off-policy actor-critic					
DDPG (2016)	deterministic AC	actor + critic	soft, $\tau = 0.001$	$Q_{\bar{\phi}}(s', \mu_{\bar{\theta}}(s'))$	first soft (Polyak) targets
TD3 (2018)	deterministic AC	actor + twin critics	soft, $\tau = 0.005$, delayed	$\min_i Q_{\bar{\phi}_i}(s', \mu_{\bar{\theta}}(s') + \tilde{\epsilon})$	clipped noise $\tilde{\epsilon}$; targets every 2nd step
SAC v1 (2018)	max-entropy AC	V network	soft, $\tau = 0.005$	$V_{\bar{\psi}}(s')$	separate V, Q and policy
SAC v2 (2018)	max-entropy AC	twin critics	soft, $\tau = 0.005$	$\min_i Q_{\bar{\phi}_i}(s', \tilde{a}') - \alpha \log \pi_{\theta}(\tilde{a}' s')$	no target actor: $\tilde{a}'$ from current π
D4PG, MPO	off-policy AC	actor + critic	periodic copy	distributional (D4PG) or Retrace (MPO)	distributed data collection
REDQ, DroQ	high-UTD AC	critic ensemble	soft	$\min_{i \in \mathcal{M}} Q_{\bar{\phi}_i}(s', \tilde{a}') - \alpha \log \pi_{\theta}(\tilde{a}' s')$	REDQ: min over a random subset
Offline RL					
BCQ, TD3+BC	offline AC	actor + twin critics	soft	clipped double-Q variants, as in TD3	stay close to the data's actions
CQL (2020)	offline AC	twin critics	soft	as SAC, plus a conservative penalty	pushes down Q on unseen actions
IQL (2021)	offline	Q network	soft	$V_{\psi}(s')$, V_{ψ} fit to $Q_{\hat{\theta}}$ by expectile regression	never queries unseen actions
Model-based					
MuZero Reanalyze	model-based	value network	periodic copy	n-step return + an older network's value	refreshes stale targets
DreamerV2	model-based	critic	periodic copy	λ -returns in imagination, from a slow critic	stabilizes value learning
Outside RL: self-supervised learning					
BYOL, MoCo, DINO	self-supervised	encoder	EMA, $m \approx 0.99$ to $0.999+$	embedding of another view of the same input	called target / momentum / teacher

Algorithms that don't use one, and why

• **REINFORCE**

Monte Carlo returns: nothing is bootstrapped, and there's no critic.

• **TRPO, PPO**

GAE targets are computed once per fresh batch and frozen: a one-batch target network in spirit.

• **GRPO, RLOO**

no critic at all: a group mean or leave-one-out average is the baseline.

• **Tabular Q-learning**

no function approximation, so updating one entry doesn't move the others.

• **A2C, A3C**

on-policy critic trained on n-step returns from fresh data.

• **IMPALA**

V-trace targets from the current network, with off-policy corrections instead of a copy.

• **DPO**

no critic; its π_{ref} is a frozen anchor for a KL term, not a source of bootstrap targets.

• **PQN, CrossQ (2024)**

removed on purpose: LayerNorm (PQN) or batch renormalization (CrossQ) keeps TD stable.

Formulas

θ^- (or a bar) marks target weights; $\text{sg}(\cdot)$ is stop-gradient. $d = 1$ only at a true terminal.

1 • The generic recipe

bootstrapped target	$y = r + \gamma(1 - d) B_{\theta^-}(s')$
loss	$L(\theta) = \mathbb{E}_{\mathcal{D}}[\ell(Q_{\theta}(s, a) - \text{sg}(y))], \quad \ell = \text{squared or Huber}$
semi-gradient	$\nabla_{\theta} L = -2 \mathbb{E}[(y - Q_{\theta}(s, a)) \nabla_{\theta} Q_{\theta}(s, a)] \quad (\ell = \text{squared})$
without a target net	B would use θ itself, so every step also moves y
The target network never receives gradients; it only changes through the update rule in card 2.	

2 • Update rules and how much they lag

hard (periodic)	$\theta^- \leftarrow \theta \quad \text{every } C \text{ steps}$
soft (Polyak / EMA)	$\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^- \quad \text{every step}$
what soft keeps	$\theta_k^- = \sum_{j \geq 0} \tau(1 - \tau)^j \theta_{k-j} \quad (\text{ignoring initialization})$
half-life	$\frac{\ln 2}{-\ln(1 - \tau)} \approx \frac{0.69}{\tau} \quad (\tau = 0.005: 138 \text{ steps})$
mean lag	soft: $\frac{1 - \tau}{\tau}$, hard: $\frac{C - 1}{2}$
matching the two	$C \approx 2/\tau: \quad \tau = 0.005 \leftrightarrow C \approx 400; \quad C = 10^4 \leftrightarrow \tau \approx 2 \times 10^{-4}$
Equal mean lag, not identical behavior: hard targets jump at each sync, soft targets drift continuously.	

3 • Fitted Q-iteration view

Bellman optimality	$(\mathcal{T}Q)(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} \left[\max_{a'} Q(s', a') \right]$
one phase	$Q_{k+1} \approx \arg \min_Q \mathbb{E}_{\mathcal{D}} \left[\left(Q(s, a) - (\hat{\mathcal{T}}Q_k)(s, a) \right)^2 \right]$

With hard updates, each C-step phase is an ordinary regression onto labels computed from $Q_k = Q_{\theta^-}$. As C grows, DQN approaches fitted Q-iteration: stable, but values propagate one Bellman step per phase.

4 • Overestimation needs its own fix

max of noisy estimates	$\mathbb{E} \left[\max_a \hat{Q}(s, a) \right] \geq \max_a \mathbb{E} \left[\hat{Q}(s, a) \right]$
Double DQN	$Q_{\theta^-}(s', \arg \max_{a'} Q_{\theta}(s', a')) \quad \text{decouples select and evaluate}$
clipped double-Q	$\min_{i=1,2} Q_{\bar{\phi}_i}(s', a') \quad (\text{TD3, SAC})$
REDQ	$\min_{i \in \mathcal{M}} Q_{\bar{\phi}_i}(s', a'), \quad \mathcal{M} \text{ a random subset of an ensemble}$
A target network slows the feedback loop; it doesn't remove the upward bias of the max.	

5 • Targets by algorithm

DQN	$y = r + \gamma(1 - d) \max_{a'} Q_{\theta^-}(s', a')$
n-step (Rainbow, R2D2)	$y = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n \max_{a'} Q_{\theta^-}(s_{t+n}, a')$
DDPG	$y = r + \gamma(1 - d) Q_{\bar{\phi}}(s', \mu_{\bar{\theta}}(s'))$
TD3 smoothing	$\tilde{a}' = \text{clip}(\mu_{\bar{\theta}}(s') + \text{clip}(\epsilon, -c, c), a_{\text{lo}}, a_{\text{hi}}), \quad \epsilon \sim \mathcal{N}(0, \tilde{\sigma}^2)$
TD3 target	$y = r + \gamma(1 - d) \min_{i=1,2} Q_{\bar{\phi}_i}(s', \tilde{a}')$
SAC v1	$y = r + \gamma(1 - d) V_{\bar{\psi}}(s')$
SAC v2	$y = r + \gamma(1 - d) \left(\min_i Q_{\bar{\phi}_i}(s', \tilde{a}') - \alpha \log \pi_{\theta}(\tilde{a}' \mid s') \right)$
IQL, value	$L_V = \mathbb{E} [L_2^{\kappa}(Q_{\hat{\theta}}(s, a) - V_{\psi}(s))], \quad L_2^{\kappa}(u) = \kappa - 1(u < 0) u^2$
IQL, Q	$y = r + \gamma(1 - d) V_{\psi}(s')$
IQL's expectile is usually called τ ; it's κ here to avoid a clash with the Polyak rate. The n-step row omits done flags for brevity.	

6 • The same trick in self-supervised learning

EMA target	$\xi \leftarrow m \xi + (1 - m) \theta, \quad m \approx 0.99 \text{ to } 0.9999$
BYOL loss	$\ \bar{q}_{\theta}(z_{\theta}) - \bar{z}'_{\xi} \ ^2_2, \quad \bar{\cdot} = \ell_2\text{-normalized}$
Convention clash: RL papers put a small τ on the online weights (0.005); BYOL, MoCo and DINO put a large m on the target (0.996 to 0.999), so $m = 1 - \tau$.	

7 • Typical settings from the papers

DQN (Nature)	hard, $C = 10,000$ updates
DDPG	soft, $\tau = 0.001$
TD3	soft, $\tau = 0.005$, updated every 2nd critic step
SAC v1 and v2	soft, $\tau = 0.005$
BYOL, DINO	$m: 0.996 \rightarrow 1$ over training (cosine schedule)
MoCo	$m = 0.999$

8 • Diagnostics

online vs. target	$\mathbb{E}[Q_{\theta}(s, a) - Q_{\theta^-}(s, a)]$
TD error	$\mathbb{E}[y - Q_{\theta}(s, a)] \quad (\text{should shrink, not explode})$
Q vs. returns	$\mathbb{E}[Q_{\theta}(s, a)]$ vs. observed discounted returns
Q values climbing without bound while returns stay flat is the signature of divergence or overestimation.	